

CLI Cheatsheet

Quick reference for the commands I always forget. Open anytime with: `open ~/bexperiments/CHEATSHEET.md` (or just ask Claude "show my cheatsheet").

Terminal multiplexer (tmux workspace)

```
./tmux-bexperiments.sh          # launch the workspace (from ~/bexperiments)
```

- One window per project, Claude auto-started in each
- `tmux attach -t bexperiments` — reattach if already running
- `NO_CLAUDE=1 ./tmux-bexperiments.sh` — launch without auto-starting Claude
- `./tmux-bexperiments.sh some-folder` — also add a window for a new project folder

Moving around inside tmux

Keys	What it does
<code>Ctrl-b</code> then <code>w</code>	window list — pick with arrows + Enter
<code>Ctrl-b</code> then a number	jump straight to that window
<code>Ctrl-b</code> then <code>n</code> / <code>p</code>	next / previous window
<code>Ctrl-b</code> then <code>d</code>	detach (everything keeps running)

Claude Code

```
claude                          # start Claude in the current folder
```

- `claude --continue` (or `-c`) — resume the most recent session in this folder
- `claude --resume` — pick an older session to resume
- `claude "fix the failing test"` — start a session with an opening prompt
- `claude -p "explain this repo"` — one-shot answer, no interactive session
- `claude --model opus` — start with a specific model
- `claude update` — update Claude Code itself to the latest version

Launching Claude in bexperiments

Remember: it's always two steps — `cd` into bexperiments first, THEN launch `claude`. Claude works in whatever folder you start it from, so if you skip the `cd` it won't see the repo. As one line:

```
cd ~/bexperiments && claude
```

```
cd ~/bexperiments/<project> && claude
```

- **Repo-wide session** (first command): sees all projects at once
- **One project only** (second command): keeps Claude focused on that folder
- **Skip both steps entirely:** `./tmux-bexperiments.sh` — every project window already has Claude running in the right folder; just switch windows and type

Slash commands (typed inside a Claude session)

Command	What it does
<code>/model</code>	switch models (Opus / Sonnet / Haiku) mid-session
<code>/clear</code>	wipe the conversation, start fresh (same folder)
<code>/compact</code>	summarize the conversation to free up context
<code>/resume</code>	switch to a different past session
<code>/cost</code>	show token usage / cost for the session
<code>/init</code>	generate or refresh the project's CLAUDE.md
<code>/memory</code>	view & edit what Claude remembers about this repo
<code>/permissions</code>	manage which tools/commands Claude may run without asking
<code>/mcp</code>	manage MCP server connections
<code>/config</code>	settings (theme, model default, etc.)
<code>/help</code>	list all commands

Keyboard shortcuts (inside a Claude session)

Keys	What it does
<code>Esc</code>	interrupt Claude mid-response
<code>Esc Esc</code>	jump back to an earlier message / rewind
<code>Shift+Tab</code>	cycle permission modes (normal → auto-accept → plan mode)
<code>!</code> at start of line	run a shell command directly (bash mode)
<code>@</code>	mention a file so it's pulled into context
<code>#</code> at start of line	save a note to memory (CLAUDE.md)
<code>Ctrl+C</code> twice	quit

Handy habits

- Start a big task in **plan mode** (`Shift+Tab` twice): Claude proposes a plan you approve before it touches files
- `#` notes are the cheap way to teach Claude repo conventions as you go
- Tell Claude "remember this" to save something across sessions

Codex (OpenAI / GPT)

Same two-step rhythm as Claude: `cd` into the folder first, then launch.

```
cd ~/bexperiments && codex      # repo-wide Codex session
```

- `codex "fix the failing test"` — start with an opening prompt
- `codex resume` — pick a previous session to resume
- `codex resume --last` — continue the most recent session (like `claude -c`)
- `codex -m <model>` — start with a specific GPT model, or type `/model` inside a session to switch
- `codex update` — update Codex itself to the latest version
- `codex login` / `codex logout` — manage the ChatGPT account it runs on
- `codex doctor` — troubleshoot if Codex is acting up (checks install, config, and auth)
- `codex exec "explain this repo"` — one-shot answer, no interactive session (like `claude -p`)

Claude ↔ Codex quick translation

Task	Claude	Codex
Launch in current folder	<code>claude</code>	<code>codex</code>
Continue latest session	<code>claude -c</code>	<code>codex resume --last</code>
Pick an older session	<code>claude --resume</code>	<code>codex resume</code>
One-shot, no session	<code>claude -p "..."</code>	<code>codex exec "..."</code>
Switch model	<code>/model</code>	<code>/model</code>
Update the tool	<code>claude update</code>	<code>codex update</code>

Turning workflows into loops & repeatable skills

Works in both Claude and Codex — and yes, plain English is the whole trick.

Loops (do this thing over and over)

Just describe it: **"turn this into a loop"**, "keep doing this every 10 minutes", "re-run this check until the tests pass". Both tools understand.

- **Claude** also has an explicit command: `/loop 5m <prompt or /command>` (e.g. `/loop 10m check if the deploy finished`). Leave off the interval and Claude paces itself. Say "stop the loop" to end it.
- **Claude scheduled runs:** ask "schedule this to run every morning at 8" — that creates a recurring cloud agent that runs even when your laptop session is closed.
- **Codex:** just ask in plain language, or for something outside a session, wrap `codex exec "..."` in a shell loop or cron job.

Skills (save a workflow so you can rerun it by name)

When you've done a workflow once and want it on tap forever, tell the tool: **"turn what we just did into a repeatable skill called "**.

- **Claude:** creates a skill you invoke by typing `/<name>` in any future session. Skills live in `.claude/skills/` (this repo only) or `~/.claude/skills/` (everywhere) — ask for whichever you want.
- **Codex:** same idea — its skills live in `~/.codex/skills/` and are invoked by name too.
- Good candidates: anything you've now explained to the agent twice.
- **Record a Skill (desktop app only):** in a Cowork task, click the + menu → **Record a Skill**. Screen-record yourself doing the workflow while narrating the important choices, and Claude turns it into a skill — no typing out steps. ~10 min limit; needs Pro/Max/Team.

Cloning a repo (copying a project from GitHub)

Same two-step rhythm: clone it, then `cd` into the folder it creates and launch Claude.

```
git clone https://github.com/<user>/<repo>.git
cd <repo>
claude
```

- The URL is on the repo's GitHub page — green **Code** button → HTTPS → copy
- `git clone` makes a new folder named after the repo, right where you ran it
- Example — this whole monorepo onto a fresh machine: `git clone https://github.com/rbpasternak-tech/bexperiments.git`
- To grab someone else's repo, clone it into your home folder (`cd ~` first), **not** inside `~/bexperiments` — it's its own git repo and nesting it inside the monorepo confuses git
- Later, `git pull` from inside the folder gets the latest updates

New project flow

Two steps: create the folder inside bexperiments, then relaunch the workspace script with the folder name — it opens a window with Claude already in the right place:

```
mkdir ~/bexperiments/my-project-name
./tmux-bexperiments.sh my-project-name
```

(Without tmux, the manual version is the same two-step idea: `cd ~/bexperiments/my-project-name && claude`)

Then jump to its window (`Ctrl-b w`) and talk to Claude. If the project sticks, add it to the `PROJECTS` list in `tmux-bexperiments.sh` and to `CLAUDE.md` / `README.md` .